

Yggdrasil ERP — Web Application Architecture

Overview

The Yggdrasil ERP web application provides browser-based access to the platform while maintaining feature parity with the native desktop client. The application is implemented using Next.js, React, and TypeScript, and communicates exclusively with the Yggdrasil C++ backend server through the public REST and WebSocket interfaces.

The web client is designed as a portable interface layer rather than the architectural reference implementation. New platform capabilities are first implemented and validated in the native desktop client and then reproduced in the web interface. This ensures that browser constraints never dictate core platform behavior while still enabling ubiquitous access across devices.

The web application exposes all primary ERP modules as well as administrative, onboarding, and portal interfaces.

Technology Stack

Component	Technology
Framework	Next.js
Language	TypeScript (strict mode)
UI Library	React
Styling	Tailwind CSS
Server State	TanStack React Query
Client State	Zustand
HTTP Client	Axios
Charting	Recharts
Validation	Zod
Testing	Vitest + Testing Library

This stack provides strongly typed client logic, predictable state management, and modular component development while maintaining compatibility with modern browser environments.

Application Structure

The web application uses Next.js App Router with filesystem-based routing. Each business module corresponds to a dedicated route within the application.

Example structure:

```
app/  
  layout.tsx  
  page.tsx  
  crm/  
  sales/  
  purchasing/  
  manufacturing/  
  warehouse/  
  finance/  
  projects/  
  plm/  
  quality/  
  service/  
  hr/  
  logistics/  
  forms/  
  approvals/  
  settings/  
  admin/  
  api-reference/  
  documents/  
  portal/
```

The `portal` section contains a simplified interface intended for account management, onboarding, documentation access, and support interactions rather than operational ERP workflows.

Rendering Model

The application uses a hybrid rendering strategy combining server and client components.

Server components handle layout composition, metadata, and static rendering tasks. Interactive functionality such as forms, tables, and live dashboards are implemented as client components using React hooks and browser APIs.

All operational data originates from the Yggdrasil backend server; the web application does not implement its own API layer.

Component Architecture

The interface is constructed from a library of reusable components that standardize interaction patterns across all modules.

Core components include:

Component	Function
CrudPanel	Generic CRUD interface for entity management
RecordDetailPage	Master-detail entity view
AttachmentPanel	File management for records
ExportButton	CSV and spreadsheet export
PrintableDocument	Print-ready document rendering
DocumentBuilder	Interactive document generation

The `CrudPanel` component acts as the primary interaction surface for module pages, providing a configurable system for data tables, forms, filtering, pagination, and row-level actions.

Chart components built with Recharts provide dashboard visualizations including bar, line, pie, and area charts.

Navigation Model

The main navigation sidebar organizes modules according to operational domains. The sidebar provides:

- module navigation
- role-based visibility
- global search
- notification indicators
- user account controls

Routes correspond directly to backend module domains, ensuring conceptual parity between interface navigation and server architecture.

State Management

The application separates server state and client state responsibilities.

Server state is managed through TanStack React Query. This library provides automatic caching, background refetching, and optimistic update capabilities for data retrieved from the backend.

Client state is handled by Zustand stores which maintain application-level information such as authentication status, user preferences, navigation state, and notification counts.

This separation prevents the common failure mode of storing server data in global client stores, which often results in stale data and manual cache management.

API Integration

All communication with the backend occurs through typed API modules located in the application's API client layer. These modules correspond directly to backend service domains.

Examples include:

• CRM • Sales • Purchasing • Manufacturing • Warehouse • Finance • Projects • PLM • Quality • Service • HR • Administration

Each module exposes strongly typed functions for list queries, entity retrieval, mutations, and workflow operations.

Axios provides HTTP communication with a shared configuration that handles authentication cookies, tenant identification headers, and automatic token refresh when authorization failures occur.

Authentication Flow

User authentication occurs through the backend authentication service.

The process follows this sequence:

1. User submits login credentials.
2. The application calls the authentication endpoint.
3. The server returns access and refresh tokens stored in secure HTTP-only cookies.
4. Client session state updates within the authentication store.
5. Navigation middleware validates session state on protected routes.

If an access token expires, the Axios interceptor automatically attempts token refresh before retrying the request.

Real-Time Integration

The web client receives live system events through WebSocket connections to the server's event hub.

These events include:

- data changes • workflow transitions • system notifications

Incoming events invalidate corresponding React Query caches, triggering automatic UI updates without polling.

Connection management includes automatic reconnection, exponential backoff, and session reauthentication after reconnection.

Validation Model

Client-side input validation uses Zod schemas which define entity validation rules shared across form components.

Client validation improves user experience by catching invalid input immediately. However, the backend server performs independent validation for all requests to ensure data integrity.

TypeScript types are inferred from Zod schemas, ensuring compile-time consistency between validation logic and application code.

Styling System

The interface uses Tailwind CSS for utility-based styling. A custom Yggdrasil color palette defines the visual identity of the platform.

Design principles emphasize:

- consistent spacing scale • responsive layouts • component-level styling • minimal global CSS

Dark-mode compatibility is supported through the Tailwind theme configuration although the feature is not currently enabled in production builds.

Portal Interface

The portal environment provides a simplified interface distinct from the operational ERP modules. It is intended for administrative and account-level interactions.

Portal features include:

- organization management • user invitations • subscription configuration • support ticket management • help documentation • onboarding workflows

This environment allows customers to manage their deployment without exposing full ERP complexity.

Export and Document Generation

The application supports two document workflows.

Data exports are generated directly within the browser, allowing users to download CSV or spreadsheet files derived from table data.

Commercial documents such as invoices, purchase orders, and quotes are generated by the backend server as formatted PDFs and delivered to the client for printing or download.

Testing and Quality Assurance

Testing is performed using the Vitest framework combined with React Testing Library.

Tests cover component behavior, state management logic, and API integration points. TypeScript strict mode provides compile-time verification of type safety across the application.

Linting rules enforce React hook correctness, import organization, and accessibility standards.

Build and Deployment

Production builds are generated using the Next.js build pipeline.

The build process produces optimized client bundles with automatic code splitting, static page generation where applicable, and compressed style assets.

Environment variables configure the backend API endpoint and WebSocket event hub address.

Continuous integration pipelines perform linting, type checking, unit tests, and production build verification before deployment.

Architectural Role

Within the Yggdrasil ecosystem, the web client serves as a universal access layer that complements the native desktop interface.

Its primary roles are:

1. Browser-based operational access
2. Responsive interfaces for tablet and mobile environments
3. Administrative and portal workflows

Because it interacts solely through the public platform APIs, the web application also acts as an external validation of the platform's interface stability and integration design.